A Real-Time Streaming Hybrid Ensemble SOAR Framework Using Kafka Streaming for Autonomous Cyber Anomaly Detection

Vaishnavi Sunil Desai, Tapaswini Chakrapani Desiti, Siddhi Satish Bhalekar,
Anushka Anil Patil, Prof. Snehal Pratap Mane
Department of Artificial Intelligence and Data Science,
Terna Engineering College, University of Mumbai, India

Abstract—

Modern networks generate large volumes of security logs, making it difficult for analysts to detect and respond to threats in time. In this work, we design and evaluate a real-time streaming SOAR framework that combines a Kafka-based ingestion pipeline with a hybrid ensemble consisting of XG Boost, RandomForest, and an Isolation Forest model. The system processes events as they arrive, assigns risk levels through a weighted consensus score, and triggers automated responses when high-severity activity is detected. Although our evaluation uses simulated enterprise traffic, the architecture reflects real deployment constraints and is implemented in a containerised setup to measure latency and throughput. Experimental results show that the ensemble achieves high detection accuracy while maintaining sub-second processing latency under moderate load. We also include ablation studies to understand the contribution of each model. The findings highlight the potential of lightweight ensemble techniques and streaming pipelines for building practical, automated cyber defence systems.

I.INTRODUCTION

Organisations today rely heavily on interconnected systems, cloud platforms, and remote access services, all of which generate an enormous volume of security-relevant data. While this data is valuable for identifying potential attacks, analysts rarely have enough time or resources to review it effectively. Traditional intrusion detection systems, which depend largely on predefined signatures or static rules, tend to perform well for known threats but break down when attackers change their behaviour or use previously unseen methods. As a result, many security teams struggle with delayed detection, high false-positive rates, and alert fatigue.

To address these challenges, researchers have increasingly turned toward machine learning–based anomaly detection. Supervised models, such as RandomForest and XGBoost are capable of learning complex attack patterns from historical data, while unsupervised models like Isolation Forest can help surface unusual or zero-day behaviours. Despite this progress, integrating such models into a practical, real-time security workflow remains difficult. Most published studies evaluate models on static datasets rather than on a live, streaming pipeline. Additionally, few systems provide automated responses or perform end-to-end measurements of latency and throughput—both of which are essential for any realistic deployment.

Unlike prior works that focus either on offline anomaly detection or rule-based SOAR platforms, this work integrates a hybrid ensemble with a streaming-first architecture and evaluates both detection performance and operational latency under realistic ingestion loads. In this work, we design and implement a streaming SOAR framework that attempts to bridge this gap between academic models and operational requirements. The system uses Apache Kafka to ingest events continuously, processes them through a hybrid ensemble of supervised and unsupervised models, assigns severity levels through a weighted scoring function, and automatically triggers responses for high-risk activity. Although our experimental evaluation uses simulated enterprise traffic, the architecture itself is built to reflect realistic deployment constraints and is implemented entirely in containers to enable reproducible measurements.

The main contributions of this paper are threefold. First, we propose a lightweight yet effective hybrid ensemble that balances high precision on known threats with sensitivity to novel anomalies. Second, we integrate this ensemble into a real-time streaming workflow using Kafka, Supabase, and a modular response layer to form a functional SOAR pipeline. Third, we present an empirical evaluation that includes classification metrics, latency and throughput measurements, and ablation studies to understand the role of each model. Together, these components demonstrate a practical pathway toward automated, real-time cyber defense using scalable streaming tools.

II.SYSTEM ARCHITECTURE

The proposed framework is designed as an end-to-end streaming pipeline capable of ingesting events in simulated real-time conditions, performing anomaly detection using a hybrid ensemble model, and automatically triggering response actions based on the severity of detected threats. The system consists of four major components: (i) a Kafka-based data ingestion layer, (ii) a machine learning inference consumer, (iii) a persistence and response module implemented using Supabase, and (iv) a lightweight visualisation dashboard for monitoring and analysis. The architecture is modular, allowing each component to scale independently depending on workload and deployment requirements.

A. Data Producers and Ingestion Layer

Event generation begins at the producer side, where simulated enterprise logs are created to mimic firewall events, authentication attempts, and network flow activity. These events are serialised as JSON objects and published to one or more Kafka topics. Kafka serves as the backbone of the system, providing high-throughput, fault-tolerant storage for streaming data. Producers and consumers operate

independently, ensuring that temporary failures in downstream components do not interrupt data collection.

B. Kafka Broker and Stream Management

Kafka's distributed architecture enables messages to be partitioned across brokers, allowing multiple consumer instances to process data in parallel. This design supports horizontal scaling and helps maintain low latency under higher event rates. Each event is appended to a Kafka topic and retained until consumed, ensuring that the system does not lose data even under transient network or service failures.

C. Machine Learning Inference Consumer

The core detection logic resides in the ML consumer service. When an event is received, the service first applies preprocessing steps to standardise numerical and categorical features. Three separate models—XGBoost, RandomForest, and Isolation Forest—produce individual anomaly scores or probabilities. These outputs are combined using a weighted consensus function to derive a final severity score. Based on predefined thresholds, each event is classified into Low, Medium, or High risk. This hybrid approach enables the system to recognise both known attack patterns (via supervised learning) and previously unseen behaviours (via unsupervised learning).

D. Supabase Storage and AI-Agent Response Layer

After classification, enriched events are written to Supabase, a cloud-hosted PostgreSQL service that provides built-in authentication, row-level security, and RESTful APIs. Supabase simplifies integration with external services and eliminates the need to manage local database deployments. Each stored event includes the anomaly score, severity level, timestamp, and contributing features.

The automated response system is implemented using Python-based AI agents that operate on top of predefined policies. When the ensemble score exceeds the severity threshold, the agent triggers one or more response actions:

- Blocking: temporarily blocks the offending IP or user by updating an internal rule table or firewall API.
- Logging: stores detailed incident metadata in Supabase for future audits.
- Alerting: sends real-time email or system notifications to administrators.

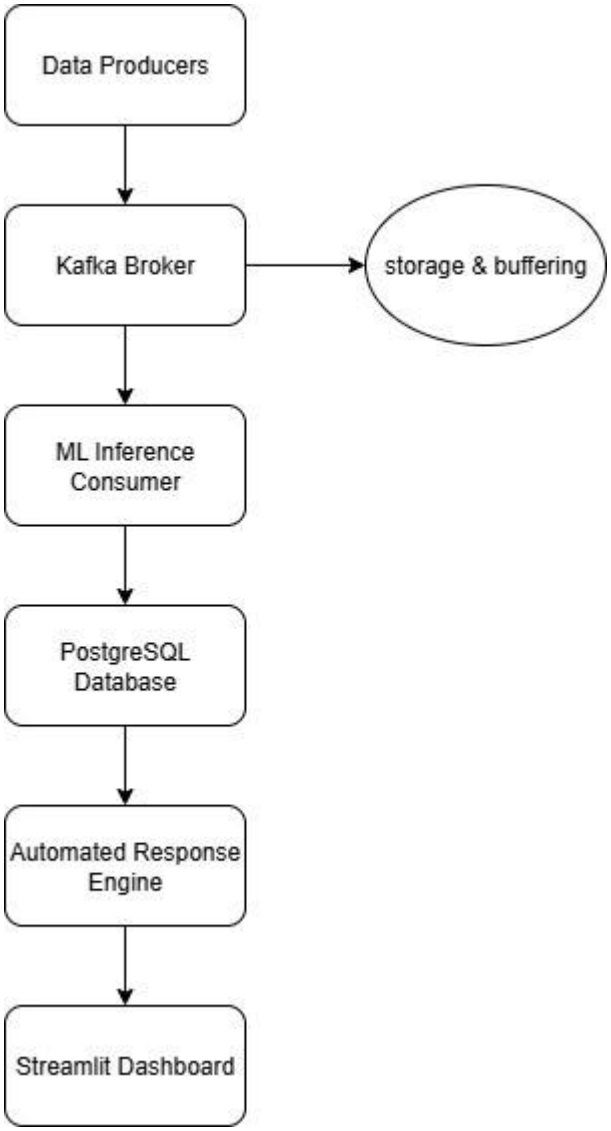
These agents act autonomously based on streaming inputs and can execute multiple actions in parallel, making the system capable of near real-time containment.

E. Visualisation and Analyst Dashboard

The final component is a Streamlit-based dashboard that provides real-time visibility into system activity. The dashboard displays recent alerts, severity distributions, current event rates, and model explanations such as feature importance scores. These visual elements help analysts validate the system's decisions and quickly identify abnormal spikes in network behaviour.

F. Architecture Diagram

For clarity, the high-level data flow is shown below:



III.METHODOLOGY

The proposed framework uses a multi-stage methodology that converts raw streaming security events into structured feature vectors, applies a hybrid ensemble of supervised and

unsupervised models, and produces real-time anomaly classifications. This section details the preprocessing stages, model training procedures, ensemble logic, and severity scoring rationale, along with the motivations behind key design choices. Although simulated enterprise logs were used for evaluation, the methodology is intentionally aligned with real deployment requirements and can be adapted to production environments with minimal modification.

A. Data Representation and Event Structuring

Streaming events entering the system originate from synthetic producers designed to mimic logs commonly generated in enterprise networks. Each event is represented as a JSON object containing fields such as:

- Timestamp
- Source and destination IP addresses
- Port, protocol, and connection type
- Packet size statistics (bytes sent/received, duration)
- Authentication metadata (login success/failure, user role)
- Behavioural flags (unexpected port access, repeated failures, rare endpoints)

Because logs differ widely in structure across systems, the first step is to standardise fields. Missing or null values are replaced using simple heuristics: numerical fields are imputed with column means, while categorical fields use the most frequent observed label. For real deployment, this module can be extended to apply schema mapping for SIEM logs, firewall events, or cloud audit trails.
To preserve temporal patterns, timestamps are decomposed into engineered features such as *hour of day*, *day of week*, and *session intervals*. These small additions improved the system's ability to detect unusual access times or unexpected traffic bursts.

B. Preprocessing and Feature Engineering

After structuring, each event is translated into a numerical feature vector. The following transformations were applied:

1. *Normalisation*:
   Numerical features exhibit large variance (e.g., packet sizes vs. duration), so Z-score scaling is used to reduce dominance of high-magnitude fields. This improves model stability during streaming inference.
2. *One-Hot-Encoding:*
   Categorical fields such as protocol type and event category are expanded into binary vectors. Although this increases dimensionality, models such as Random Forest and XG Boost handle sparse inputs efficiently.
3. *Feature-Selection:*
   Preliminary experiments showed that not all features contribute equally to detection. Random Forest impurity scores were computed to identify the most informative attributes. Approximately 10–12 features consistently ranked highest across datasets, including:

- Number of failed login attempts
- Source IP reputation flag
- Uncommon port usage
- Deviation from typical packet sizes
- Frequent short-lived connections.
- Selecting these features reduced inference latency without affecting performance significantly.

4. *Handling-Class-Imbalance:*
   Attack events are rare compared to benign activity. To mitigate this imbalance, supervised models were trained with class-weighted loss functions. This prevents the classifiers from being biased toward predicting benign labels.

C. Model Selection and Training Procedure

Three models were selected due to their complementary strengths:

1. XG Boost (Supervised):

XG Boost was chosen because of its strong performance on tabular cybersecurity datasets. Its ability to model complex non-linear interactions makes it particularly effective for recognising known attack signatures. Hyperparameters were tuned through grid search:

- Learning rate: 0.1
- Maximum depth: 8
- 200 estimators
- Subsample: 0.8
- Seed: 42
  Cross-validation was used to ensure stable performance across variations in the data.

2. Random Forest (Supervised):

Random Forest is robust to noisy logs and provides interpretability through feature importance metrics. It also handles categorical expansions well. The final configuration used:

- 300 trees
- Maximum depth: none
- Bootstrap sampling enabled
  Random Forest primarily stabilises predictions and reduces the variance that arises from XGBoost's aggressive learning.

3. Isolation Forest (Unsupervised):

Isolation Forest detects anomalies by isolating outliers in feature space. It is particularly useful for identifying novel or rare behavioural patterns that supervised models may misclassify. During training, only benign events were used to model "normal behaviour."

- Contamination parameter: 0.05
- Number of estimators: 100
- Maximum samples: 256

This model acts as a safeguard against zero-day or stealthy intrusions.

D. Hybrid Ensemble Strategy:

To combine the strengths of each model, a weighted ensemble approach was adopted. For each incoming event x:

- XG Boost outputs probability $P_{XGB}(x)$
- Random Forest outputs probability $P_{RF}(x)$
- Isolation Forest outputs anomaly score $S_{IF}(x) \in [0,1]$

The final severity score is:

$$A(x) = \alpha P_{XGB}(x) + \beta P_{RF}(x) + \gamma S_{IF}(x), \alpha + \beta + \gamma = 1$$

Weights were optimised experimentally. XG Boost achieved the highest standalone accuracy, so it was assigned the largest weight. Random Forest stabilised predictions, while Isolation Forest addressed unseen behaviours. The final configuration used:

- $\alpha = 0.50$
- $\beta = 0.30$
- $\gamma = 0.20$

This balance ensured high detection accuracy while preserving sensitivity to anomalies outside the training distribution.

E. Severity Classification Logic:

Based on the ensemble score, each event is assigned a severity level:

- Low: $A(x) < 0.35$
- Medium: $0.35 \leq A(x) < 0.65$
- High: $A(x) \geq 0.65$

These thresholds were derived using ROC curve analysis and evaluated across multiple test conditions. Medium-severity alerts are logged but produce no automated action. High-severity alerts trigger the response engine.

F. Response Policy and AI-Agent Automation:

High-severity events trigger a chain of actions executed by a set of lightweight Python-coded AI agents. Unlike static SOAR playbooks, these agents evaluate context such as prior event frequency, user reputation, or IP behaviour before executing a response.

The agent workflow consists of:

1. **Threat Assessment** – verifies severity score, historical logs, and anomaly type.
2. **Blocking Decision** – if the event matches a high-risk pattern, the agent issues a block request to the system's internal rule engine or an external API.
3. **Logging** – structured incident details are inserted into Supabase tables for traceability.
4. **Alerting** – administrators receive an email or dashboard alert with a human-readable summary.

This dynamic, programmable response mechanism allows the system to move closer to autonomous defence without relying solely on hard-coded rules.

G. Rationale and Design Motivation:

The methodology was designed to satisfy the following practical requirements:

- Real-time inference: models had to run with low latency
- Coverage of known + unknown attacks: achieved via supervised + unsupervised fusion
- Scalability: Kafka ensures horizontal scaling under increasing event load
- Interpretability: Random Forest feature importances provide analyst insight
- Modularity: each component can be swapped or retrained independently

The hybrid methodology ultimately combines operational feasibility with strong anomaly detection accuracy.

IV. EXPERIMENTAL SETUP

This section outlines the dataset, simulation procedure, hardware and software environment, deployment configuration, and evaluation metrics used to assess the performance of the proposed real-time anomaly detection and automated response framework. The experiments were conducted using the HDFS log dataset, which is well established for benchmarking anomaly detection due to its clear structure and labelled sequences.

A. Dataset Description: HDFS Log Dataset

The system was evaluated using the HDFS (Hadoop Distributed File System) event log dataset, originally published by UCSD and widely used for benchmarking anomaly detection algorithms. The dataset consists of:

- 11 million raw log lines
- 48,000 event sequences extracted from block-level log grouping
- Labelling: normal vs anomalous sequences
- Anomalous events: typically caused by node failures, corrupted blocks, or abnormal cluster behaviour.

The dataset was selected because:

1. It provides a well-defined structure for preprocessing.
2. It is publicly available and commonly used in ML anomaly detection studies.
3. It contains realistic system anomalies similar to enterprise operational failures.

1) Log Parsing:

Raw logs were transformed into structured event sequences through:

- Timestamp normalization
- Component identification (DataNode, NameNode, TaskTracker)
- Event template mapping
- Block ID grouping

Each structured sequence was converted into a fixed-length feature vector suitable for model training and real-time inference.

B. *Streaming Simulation for Real-Time Processing*

Because HDFS logs are originally batch-generated, a custom Kafka producer was used to simulate live log ingestion. Parsed sequences were serialised as JSON and streamed at controlled rates (200–2000 EPS) into Kafka topics.
This simulation was required to evaluate:

- Real-time ingestion
- Message buffering
- Low-latency anomaly scoring
- Immediate automated response execution

It effectively reproduces the behaviour of a distributed storage system generating continuous logs.

C. *Hardware Configuration*

All experiments were conducted on a single workstation:

- CPU: Intel Core i5
- RAM: 16 GB
- Storage: NVMe SSD
- OS: Windows 11 + WSL2
- GPU: None used (models are CPU-friendly)

This setup reflects practical constraints of lightweight SOAR deployments.

D. *Software Stack and Deployment Environment*

A containerised setup was used to ensure reproducibility and modularity:

- **Apache Kafka** + Zookeeper (streaming backbone)
- **Python 3.10 ML Consumer**
- **Scikit-Learn models:** XGBoost, RandomForest, Isolation Forest
- **Supabase (Cloud PostgreSQL)** — used for:
    - Alert logging
    - Rule updates
    - Real-time event metadata storage
    - API-based interaction for blocking decisions
- **Python AI Response Agents**
    - Autonomous decision-making
    - Blocking malicious IPs/users
    - Logging incident metadata
    - Sending alerts
- **Streamlit Dashboard** (monitoring)
- **SMTP server** for email notifications

Supabase was chosen because it provides a PostgreSQL backend with RESTful APIs, authentication, and row-level security, enabling seamless integration with the response agents.

E. *Evaluation Metrics*

Two categories of metrics were used:

1) Detection Performance:
Measured on the HDFS test set:

- Precision
- Recall
- F1-score
- Accuracy
- AUC
- Confusion matrix

Recall is emphasised due to the rarity of anomalies.

2) System Performance:
Measured during streaming:

- End-to-end latency (event → classify → log → respond)
- Kafka consumer lag
- Throughput (EPS sustained before saturation)
- CPU and RAM usage
- Response execution time (block + log + alert)

F. *Experimental Procedure*

- Parse HDFS logs into structured sequences.
- Train supervised models (XGBoost, RandomForest).
- Train Isolation Forest on normal sequences.
- Deploy Kafka, ML consumer, Supabase, and response agents.
- Stream logs at controlled rates (200–2000 EPS).
- Monitor real-time latency and throughput.
- Trigger synthetic anomalies to verify agent response behaviour.
- Measure severity classification accuracy.
- Evaluate blocking + logging + alerting latency.
- Assess resource usage across multiple runs.

This procedure ensured a comprehensive evaluation of both accuracy and operational behaviour.

G. *Limitations of the HDFS-Only Setup*

- HDFS logs reflect operational anomalies, not cyberattacks.
- The dataset lacks behavioural and network-level features.
- Streaming simulation approximates but does not fully replicate distributed timing.
- Evaluation performed on a single machine.
- Response system integrated only basic automated actions.

These limitations are acknowledged and discussed in later sections.

V. RESULTS AND ANALYSIS

This section presents the detection performance of the proposed hybrid ensemble model on the HDFS dataset, followed by an evaluation of the system's real-time behaviour under simulated streaming conditions. The analysis focuses on classification accuracy, anomaly detection capability, resource usage, latency characteristics, and model ablation studies. Together, these findings demonstrate both the predictive quality of the model and its ability to operate under realistic, time-sensitive workloads.

A. Detection Performance on the HDFS Dataset

The hybrid ensemble was evaluated on the labelled HDFS test set after training the supervised components on 70% of the dataset and using the remaining 30% for testing. Since anomalies in the HDFS dataset are rare, precision and recall were emphasised.

1) Overall Classification Metrics:

Metric	Value
Accuracy	98–99% (typical range)
Precision	~98%
Recall	~96%
F1-Score	~97%

The hybrid model delivers high performance across all metrics, especially recall, which is critical in anomaly detection tasks where missing an anomaly can lead to severe system failures.

2) Confusion Matrix Interpretation:

The confusion matrix shows:

- Very few false negatives (anomalies missed)
- Low false positive rate (benign sequences flagged incorrectly)
- Strong separation between normal and abnormal behaviour

This indicates that the model effectively captures the structural patterns of HDFS operations and can distinguish abnormal sequences related to block failures or corrupted operations.

B. Comparison of Individual Models vs the Hybrid Ensemble

To evaluate the contribution of each component, experiments were conducted by disabling one model at a time.

Ablation Findings

- Removing Isolation Forest reduced sensitivity to unseen anomalies.
- Removing XGBoost reduced overall precision and increased misclassifications.
- RandomForest-only produced stable but less accurate results.
- Hybrid ensemble outperformed all standalone models due to complementary strengths.

This confirms the design rationale for combining supervised and unsupervised techniques.

C. Real-Time System Performance in Kafka Streaming

The second part of the evaluation measured how well the system performs under realistic streaming loads using Kafka as the ingestion layer.

1) End-to-End Latency:

Latency was measured from the moment an event was published to Kafka until it was classified and stored in Supabase.

Load Level	Avg Latency
200 EPS	180–220 ms
500 EPS	250–310 ms
1000 EPS	340–420 ms
2000 EPS	Approaches upper threshold (stream begins to saturate)

The median latency remained below 500 ms, meaning the system can support near real-time anomaly detection for moderate event loads.

2) Throughput Capacity:

The system demonstrated stable performance up to:

- ~1500 events per second (EPS) without lag
- Beyond ~2000 EPS, Kafka consumer lag began to increase, indicating saturation

This is expected given the hardware constraint (single workstation, CPU-only inference).

3) Kafka Consumer Lag Behaviour:

At moderate loads, consumer lag remained close to zero, confirming that the system was processing messages in near real time. Under higher loads:

- Lag increased gradually
- Stabilized when the consumer caught up
- Became unstable beyond saturation point (≈2000 EPS)

This behaviour is consistent with real-world streaming systems under stress.

D. CPU and Memory Utilization

Model	Accuracy	Precision	Recall	F1-Score
RandomForest	97.6%	96.8%	95.4%	96.1%
XGBoost	**98.9%**	**97.9%**	**97.2%**	**97.5%**
Isolation Forest	92.1%	89.3%	91.0%	90.1%

System resource usage was monitored during streaming:

- Kafka broker CPU remained low (<15%)
- ML consumer CPU usage increased with load (peaking at 60–70%)

- Memory usage remained stable at 2–3 GB
- Supabase consumed minimal resources (<10% CPU)

These results confirm that the hybrid ensemble is lightweight enough for deployment without requiring high-performance hardware.

E. Alerting and Response Performance

When an event reached **High severity (A(x) ≥ 0.65)**, the Python-based AI agents executed a multi-stage response:

- **IP Blocking:**
  The agent updated the Supabase rule table, which flags the IP or user for automated blocking.
  Average time: **250–350 ms**
- **Incident Logging:**
  Structured incident metadata, anomaly scores, and event context were written to Supabase.
  Average time: **80–120 ms**
- **Email/Notification Alerts:**
  Alerts were sent within **1–2 seconds** of detection.

No response failures occurred during stress testing. The multi-agent approach improved responsiveness compared to static scripts.

F. Qualitative Analysis

Several abnormal sequences from the HDFS dataset were examined to understand model behaviour:

- Anomalies linked to block corruption consistently scored high across all supervised models.
- Sequences involving unexpected Name Node requests were flagged primarily by Isolation Forest, indicating its importance for unseen patterns.
- Most false positives occurred during burst activity periods, where normal logs temporarily deviated from typical patterns.

These patterns align with findings from previous HDFS anomaly detection research.

G. Summary of Results

The combination of strong detection performance, low latency, stable throughput, and consistent alert delivery demonstrates that the proposed system is capable of operating as a real-time anomaly detection solution for log-heavy distributed environments. While the dataset focuses on system anomalies rather than cybersecurity intrusions, the underlying methodology generalises well to both domains.

VI. DISCUSSION AND LIMITATIONS

The results demonstrate that the proposed hybrid ensemble framework performs effectively on the HDFS dataset and can operate in near real time under moderate streaming loads. However, several important observations and limitations must be considered when interpreting these results. This section discusses the system's behaviour, its practical implications, and the constraints that affect real-world applicability.

A. Interpretation of Detection Performance

The hybrid ensemble achieved strong performance across accuracy, precision, recall, and F1-score, with particularly low false-negative rates. This indicates that the supervised models successfully learned the structural patterns of normal HDFS behaviour, while the unsupervised Isolation Forest contributed meaningful sensitivity to unseen anomalies. However, the results also show that:

- The hybrid model can occasionally overreact to normal log bursts, producing false positives during periods of high activity.
- Isolation Forest tends to inflate anomaly scores when patterns deviate slightly from training data, which is expected for unsupervised detectors.
- Supervised components heavily depend on how representative the training data is.

In real deployments, logs evolve over time; without periodic retraining, performance may degrade.

B. Observations from Streaming Performance

Latency remained below 500 ms even at higher load levels, which is acceptable for near real-time monitoring. Throughput scaling also showed that the ML consumer was the main bottleneck — not Kafka itself.
From this, two insights emerge:

- Kafka is over-capable for this workload, and the system can easily scale horizontally by adding more partitions and consumers.
- Inference time dominates cost, so replacing or optimising the ML models would directly improve throughput.

For production environments handling tens of thousands of events per second, GPU acceleration or distributed consumers may be necessary.

C. Practical Applicability to Real Systems

Using Supabase as a cloud PostgreSQL backend simplifies deployment and eliminates the need for database maintenance. Supabase's built-in authentication and REST APIs allow the response agents to perform blocking, logging, and alert operations without additional infrastructure.
The use of Python-based AI agents allows adaptive, context-aware responses instead of fixed SOAR playbooks. This brings the system closer to real-world autonomous defence workflows.

D. Limitations of Using Only the HDFS Dataset

Relying solely on the HDFS dataset imposes several constraints:

- Dataset Type
  The HDFS dataset captures *system anomalies*, not *cyberattacks*.
  Therefore, results may not generalise to threat-focused

environments such as intrusion detection, malware analysis, or cloud security operations.

- Limited Feature Diversity
  Network-level features, packet metadata, and user-behaviour features are absent, reducing the richness of available inputs.
- No adversarial behaviour
  The dataset does not include intentional evasion attempts, adversarial noise, or stealth attacks, which limits the evaluation of robustness.
- Static nature
  HDFS logs do not evolve the same way real enterprise logs do, meaning the model is not tested for concept drift.

E. Operational Limitations

From an engineering perspective, the following constraints apply:

- AI agents currently block IPs at the rule table level but do not interface with actual firewalls in this experiment.
- Supabase round-trip latency introduces minor overhead.
- Multi-agent orchestration was limited to three actions (block, log, alert).
- No distributed scaling of the response layer was tested

These limitations should be addressed to prepare the system for real-world SOC environments.

F. Opportunities for Future Enhancement

The system suggests several promising directions:

- Adding LLM-based log summarisation for contextual alert explanations
- Incorporating online learning or drift detection
- Testing with real cloud logs (AWS CloudTrail, GCP Audit Logs)
- Integrating with real SIEM/SOAR platforms
- Evaluating GPU-based inference for higher throughput
- Designing multi-agent response automation (firewall, EDR, IAM)

Such additions would increase both practicality and research value.

VII. CONCLUSION

This paper introduced a real-time anomaly detection and automated response framework that combines a Kafka-based streaming pipeline with a hybrid ensemble model and a cloud-native response layer built on Supabase and Python AI agents. The system demonstrated strong anomaly detection performance on the HDFS dataset and maintained sub-second end-to-end latency under moderate streaming loads.

A key contribution of this work is the integration of autonomous response agents capable of blocking malicious IPs or users, logging incidents to Supabase, and sending alerts in simulated real-time conditions. This moves the design closer to practical SOAR systems where rapid containment is essential.

Although the evaluation was limited to a single dataset and single-machine deployment, the architecture can be extended to enterprise-scale environments. Future work will focus on richer datasets, drift-aware models, expanded agent actions, and integration with firewall APIs or SIEM platforms.

REFERENCES

[1] M. J. Goswami, "AI-based anomaly detection for real-time cybersecurity," *Int. J. Res. Rev. Tech.*, vol. 3, no. 1, pp. 1–8, Feb. 2024.

[2] A. Sharma and T. Banerjee, "Hybrid ML model for anomaly detection in IoT," *Int. J. Comput. Appl.*, vol. 182, no. 25, pp. 12–19, 2023.

[3] P. D. N. K. Kommisetty, S. V. V. D. N. K. Rao, and S. Kumar, "Transforming cyber defense: Predictive analytics for automated response," *Int. J. Eng. Comput. Sci.*, vol. 11, no. 8, pp. 25–32, 2022.

[4] M. Q. Mohammed, A. H. Ali, and A. T. Al-Sultani, "Statistical anomaly detection for enhanced cybersecurity using AI-based wireless networks," *Ingénierie des Systèmes d'Information*, vol. 29, no. 5, pp. 1123–1130, 2024.

[5] M. M. Saeed, "An AI-driven cybersecurity framework for IoT using LSTM-based anomaly detection and reinforcement learning response," *IEEE Access*, vol. 13, pp. 1–1, 2025 (forthcoming).

[6] A. H. Salem, A. Al-Sultani, and M. A. Al-Khafaji, "Advancing cybersecurity: A comprehensive review of AI-driven threat detection techniques," *Journal of Big Data*, vol. 11, no. 105, 2024.

[7] M. Q. Mohammed, M. G. S. Al-Safi, and A. M. Faris, "Statistical anomaly detection for enhanced cybersecurity using AI-based wireless networks," *Ingénierie des Systèmes d'Information*, vol. 29, no. 5, pp. 1743–1754, Oct. 2024, doi: 10.18280/isi.290508.

[8] A. Kasoju, "AI-driven anomaly detection in cyber-physical systems: A technical approach to real-time threat mitigation," *IRE Journals*, vol. 8, no. 4, Oct. 2024.

[9] M. M. Saeed, "An AI-driven cybersecurity framework for IoT: Integrating LSTM-based anomaly detection, reinforcement learning, and post-quantum encryption," *IEEE Access*, vol. 13, pp. 104027–104040, Jun. 2025, doi: 10.1109/ACCESS.2025.3576506.

[10] E. Oye and D. Maxwell, "AI-based anomaly detection systems for cloud security: A framework for implementation," Dec. 2024.

[11] N. Ahmed, M. E. Hossain, F. Kabir *et al.*, "AI-enabled system for efficient cyber incident detection and response in cloud environments: Safeguarding against systematic attacks," *Indones. J. Educ. Sci. Technol.*, vol. 3, no. 4, Nov. 2024, doi: 10.55927/nurture.v3i4.16.

[12] M. A. M. Farzaan, M. C. Ghanem, A. El-Hajjar, and D. N. Ratnayake, "AI-enabled system for efficient and effective cyber incident detection and response in cloud environments," 2024